

Maritime Chartering Decision Platform

Complete technical documentation — the data layer, the five analytical modules, the mathematics behind every number, the API surface, the frontend, and a candid assessment of what the system does and does not yet prove.

6

Analytical modules wired into one recommendation

35

Modelled origin → East Coast trade lanes

52

Engineered features per route

1,000

Monte Carlo rate paths per decision

SUBMITTED BY

Team IIT ISM Dhanbad

CLIENT

SAIL — Steel Authority of India Ltd.

STACK

FastAPI · XGBoost · LightGBM · LSTM

DOCUMENT

v1.0 — Full System Reference

Table of Contents

Fifteen sections. Sections 1–3 are readable by anyone. Sections 4–11 contain the full mathematics and are the reference for reviewers. Section 14 is the honest limitations register and should be read before any deployment decision.

1	Executive Summary What the platform is and what it answers	9	Module 5 — Charter vs Spot Engine NPV, breakeven, Monte Carlo, VaR
2	Problem Statement — PS-26006 SAIL's five chartering questions	10	Module 6 — Risk Summary Weighted composite risk scoring
3	System Architecture Three tiers, data flow, repository map	11	API Layer Ten endpoints, schemas, error handling
4	The Data Layer Synthetic generator, 52 features, 35 lanes, 7 vessels, 33 ports	12	Frontend Dashboard Components, charts, design system
5	Module 1 — Freight Rate Forecasting XGBoost + LightGBM + LSTM ensemble with quantile CIs	13	End-to-End Worked Example One parcel traced through all six modules
6	Module 2 — Vessel Optimizer Feasibility gates, scoring, multi-voyage planning	14	Limitations & Validity Register What is real, what is stubbed, what must be fixed
7	Module 3 — Delivered Cost Engine Eight cost components beyond the freight quote	15	Deployment, Reproduction & References Run instructions, environment notes, citations
8	Module 4 — Lightering Optimizer TPC curve integration, STS costing, weather gating		

READ THIS FIRST

Every freight-rate history, port draft limit, berth-availability figure and weather probability in this repository is **synthetic demo data**, generated by a seeded simulator in [backend/data/generate_synthetic_data.py](#). It is not Baltic Exchange data, not Clarksons data and not live port information. The models, the engineering and the decision logic are real and inspectable; the *numbers they are fitted to are placeholders*. Section 14 quantifies exactly how far the current outputs can be trusted.

1. Executive Summary

What the platform is, the five questions it answers, and why it is built the way it is.

1.1 The one-paragraph version

A chartering desk at SAIL receives a parcel of coal or steel, an origin region and an East Coast discharge port. Historically the answer came from a broker's phone call and a spot fixture. This platform replaces that with a single HTTP call that runs six analytical modules in sequence, each one consuming the previous one's output, and returns one connected recommendation: *which ship, at what delivered cost, chartered or spot, entered when.*

1.2 The five questions, and the module that answers each

#	QUESTION	ANSWERED BY	OUTPUT
1	When should we charter?	models/forecasting.py	90-day rate forecast with 80% and 95% quantile confidence bands
2	What does the voyage truly cost?	models/cost_calculator.py	Eight-component delivered cost, not just the freight quote
3	Which ship can actually do it?	models/vessel_optimizer.py	Feasible vessel chosen from parcel size + both ports' draft, LOA, beam and handling limits — or a multi-voyage plan
4	What if the ship is too deep?	models/lightering.py	TPC-integrated lightering tonnage, STS cost, weather-gated recommendation
5	Charter or spot?	models/decision_engine.py	NPV comparison, breakeven rate, 1,000-path Monte Carlo, 95% VaR, timing and idle signals
—	How risky is the whole plan?	main.py :: _calculate_risk_summary	Weighted 0–100 composite across weather, congestion and volatility

The user never selects a vessel class. The optimizer derives it, and every downstream module then consumes that derived vessel — this is the single most important architectural decision in the system.

1.3 Why the modules are connected rather than separate

Most "AI" tools in this space return a rate number and stop. The modules here are chained so that each one narrows the problem for the next: the **optimizer** decides the parcel size per voyage, which sets the **forecaster's** cargo basis; the **forecaster's** day-1 rate becomes the **cost engine's** freight input; the optimizer's chosen class becomes the **lightering** module's hull; and the forecast's confidence band becomes the **decision engine's** simulation variance. Change the parcel and the entire chain re-derives.

1

Constraint-first

Vessel feasibility is resolved *before* economics. A 180,000 t Capesize on Newcastle→Paradip is rejected on LOA and beam at Newcastle, not scored and rejected on price.

2

Full delivered cost

Freight is typically 60–65% of delivered cost. Judging a charter on the freight quote alone is judging 35% of the decision.

3

Explicit uncertainty

Every forecast carries a distribution, not a point. The Monte Carlo propagates that distribution into a probability and a VaR, so the desk sees the downside before committing.

1.4 Headline measured results (AUS_PAR, 120,000 t coal, June loading)

These are outputs from the running system, reproducible with the commands in Section 15. They illustrate behaviour and magnitudes; they are **not** market forecasts (see Section 14).

\$25.07

DELIVERED COST / TONNE

13.36

BREAKEVEN CHARTER RATE \$/T

93.3%

P(CHARTER CHEAPER)

MODERATE

COMPOSITE RISK 52/100

2 × Panamax

0.0357

–\$27k

0 / 35

KAMSARMAX, 60 KT EACH

MODEL MAE (\$/T)

VAR 95% ON CHARTER

LANES WITH CANAL TRANSIT

2. Problem Statement — PS-26006

Why SAIL asked for this, and the operational pain it replaces.

2.1 Context

SAIL procures bulk raw material — coking coal and steel — on long-term contracts from overseas origins (Australia, the US Gulf, Mozambique, Russia, Indonesia) into Indian East Coast discharge ports. The freight and chartering decision sits between the commercial contract and the physical voyage. The stated problem is that chartering is **reactive and one-off**: each fixture is negotiated in isolation, at the moment cargo is ready, with no forward view of where rates are heading.

2.2 The five operational pain points, and where each is addressed

PAIN POINT	CONSEQUENCE	ADDRESSED IN
No forward rate view	Fixtures are priced off yesterday's market. A buyer with no view of a coming soft patch overpays.	\$5 Quantile-forecast ensemble
Freight-quote tunnel vision	Port charges, waiting time, demurrage, weather, insurance and bunkers are discovered <i>after</i> the fixture is signed.	\$7 Eight-component cost engine
Manual vessel matching	Broker proposes whatever is in the market. A ship that cannot physically enter the discharge berth is offered anyway.	\$6 Constraint-first optimizer
Draft surprises	A loaded ship arrives at a shallow East Coast port needing ship-to-ship transfer, discovered at anchor.	\$8 TPC-integrated lightering
Idle time accumulation	Between parcels the vessel waits. Nobody quantifies the low-demand windows or suggests alternative employment.	\$9.6 Operational insights
Charter vs spot by gut feel	Long-term charter commitments are made without quantifying the probability of regret.	\$9 NPV + Monte Carlo + VaR

2.3 Scope boundary — what the problem statement does *not* ask for

- **No live market feed.** The platform consumes a modelled rate series. Wiring a Baltic Exchange or SAIL approved feed is an integration task, not a modelling one — the forecaster's input contract is `features_<ROUTE>.csv`.
- **No cargoes-fixture matching algorithm.** The platform optimises the *transport* decision downstream of a cargo contract, not the counterparty selection itself.
- **No legal or charter-party drafting.** Time charter versus voyage charter is modelled purely as a discounted cash-flow comparison of the freight leg.

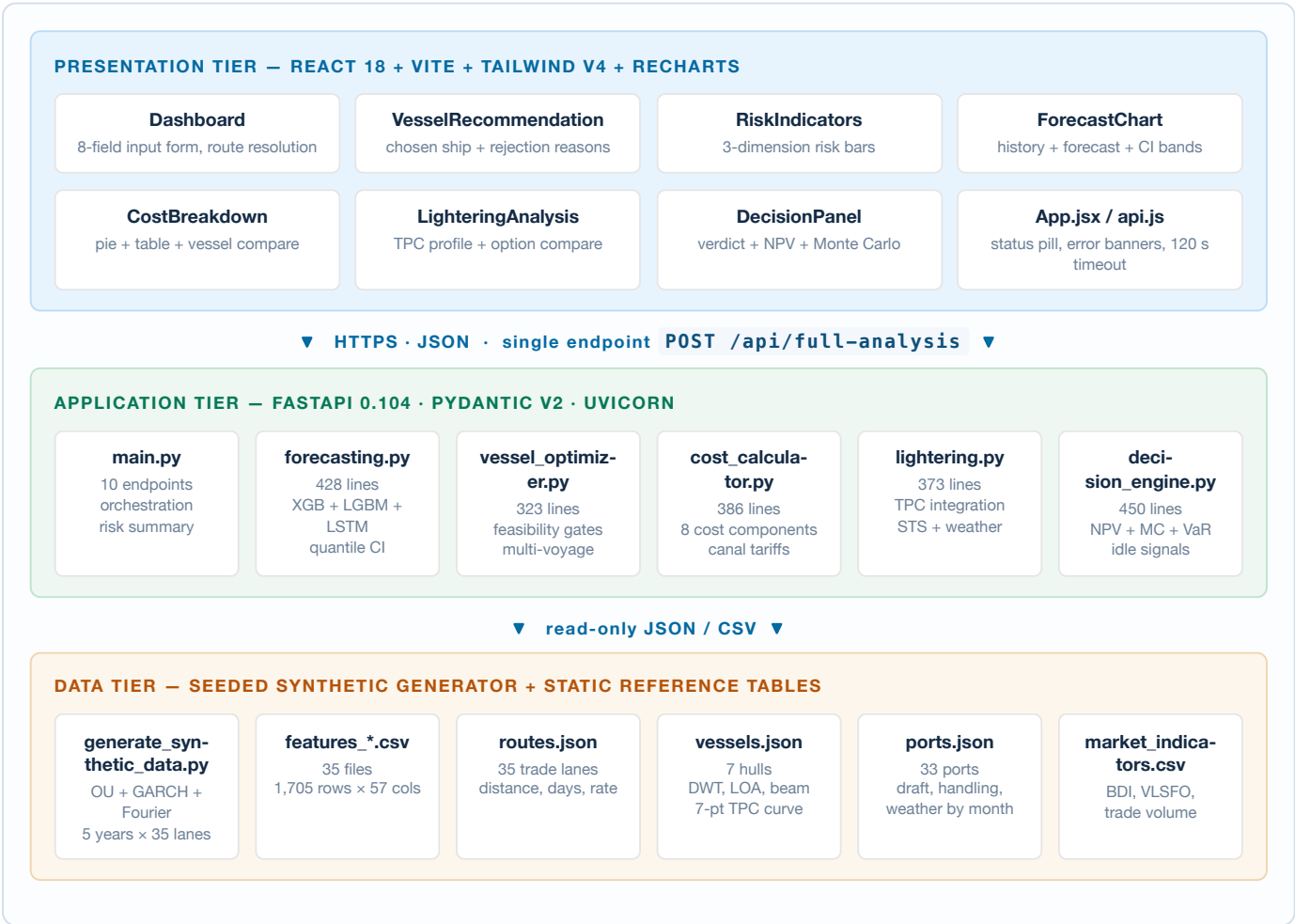
DESIGN CONSEQUENCE

Because the rate feed is modelled, every downstream module is written against a *stable interface* rather than against a specific data vendor. Replacing the synthetic series with approved data is a file substitution plus a re-run of the generator, not an architectural change. This was a deliberate design choice, and it is also the reason the current numerical outputs must not be presented as market intelligence.

3. System Architecture

Three tiers, a strictly one-directional data flow, and a single orchestration endpoint.

3.1 Logical architecture



3.2 Request lifecycle of POST /api/full-analysis

The orchestration order in `main.py::full_analysis` is load-bearing, not incidental:

STEP	ACTION	WHY IT MUST BE HERE IN THE ORDER
1	<code>recommend_vessel()</code>	Determines class, subtype and per-voyage tonnage . Every cost, lightering and decision number downstream is scoped to this parcel, not the original request tonnage.
2	<code>get_forecast(90d)</code>	Produces the 90-day rate path. Day-1 value becomes the freight rate input to step 3. Cached per route in <code>_forecaster_cache</code> .
3	<code>calculate_total_cost()</code>	Uses the step-1 vessel and the step-2 rate. If <code>split_voyages > 1</code> , the per-voyage total is multiplied up and cost/tonne recomputed against the original parcel.
4	<code>compare_vessel_options()</code>	Re-invokes the optimizer internally, then prices only the hulls that passed. Guarantees the comparison never shows an infeasible ship.
5	<code>full_lightering_analysis()</code>	Evaluates the step-1 hull against the discharge port only, since lightering is a discharge-side problem.
6	<code>charter_decision()</code>	Calls the same cached forecast (step 2), aggregates days into contract months, runs NPV + breakeven + Monte Carlo.
7	<code>_calculate_risk_summary()</code>	Reads weather risk (step 5), waiting days (step 3) and CI width (step 2) and combines them.

Each step is individually wrapped in `try/except` and its failure is appended to a `warnings` array rather than aborting the request. A partial analysis with warnings is returned instead of a 500.

3.3 Technology stack and the reason for each choice

LAYER	TECHNOLOGY (PINNED VERSION)	WHY
Gradient boosting	XGBoost 2.0.3	Native <code>reg:quantileerror</code> objective — this is the single reason XGBoost is the CI engine. Mean-squared-error models have no honest interval.
Gradient boosting	LightGBM 4.2.0	Second, structurally different tree learner. Diversifies the ensemble more than a second XGBoost seed would.
Sequential	PyTorch 2.1.2 (LSTM)	Captures residual autocorrelation the trees smooth out. Deliberately restricted to horizons ≤ 7 days.
Preprocessing	scikit-learn 1.3.2	<code>StandardScaler</code> , <code>TimeSeriesSplit</code> , MAE/RMSE/MAPE metrics.
Numerics	NumPy 1.26.2 · SciPy 1.11.4 · pandas 2.1.4	Vectorised simulation and time-series assembly.
API	FastAPI 0.104.1 · Pydantic 2.5.3 · Uvicorn 0.24.0	Request validation with range constraints (<code>ge</code> , <code>le</code> , <code>gt</code> , <code>lt</code>) rejects bad input before any model runs.
Frontend	React 18.2 · Vite 6 · Tailwind CSS 4 · Recharts 2.10	Vite dev proxy forwards <code>/api</code> to <code>:8000</code> , so no CORS preflight in development.
Icons / HTTP	lucide-react 0.294 · Axios 1.6.2	Axios timeout set to 120 s to absorb first-call model training.

3.4 Repository map

```
├── backend/
│   ├── main.py                # 490 lines – FastAPI app, 10 endpoints, orchestration, risk summary
│   ├── requirements.txt
│   ├── models/
│   │   ├── forecasting.py     # 428 lines – FreightLSTM, FreightForecaster, get_forecast()
│   │   ├── vessel_optimizer.py # 323 lines – recommend_vessel() single + split voyage
│   │   ├── cost_calculator.py # 386 lines – calculate_total_cost(), compare_vessel_options()
│   │   ├── lightering.py      # 373 lines – TPC integration, STS cost, weather risk
│   │   └── decision_engine.py  # 450 lines – NPV, breakeven, Monte Carlo, operational insights
│   └── data/
│       ├── generate_synthetic_data.py # 318 lines – OU + GARCH + Fourier generator
│       ├── routes.json           # 35 PS-26006 trade lanes
│       ├── vessels.json          # 7 hulls with 7-point TPC curves
│       ├── ports.json           # 33 ports: draft, LOA, beam, handling, weather
│       ├── freight_rates.csv     # 1,825 days × 35 rate series
│       ├── market_indicators.csv # BDI, VLSFO, trade volume index
│       └── features_<ROUTE>.csv  # 35 files × 1,705 rows × 57 columns
├── frontend/
│   ├── src/App.jsx             # header, health probe, status pill, offline/no-data banners
│   ├── src/api.js              # Axios instance, 120 s timeout, error interceptor
│   └── src/components/         # 7 components – see Section 12
├── run.sh / run.bat            # one-command bootstrap for Linux/macOS and Windows
└── README.md · CHANGELOG_PS26006.md
```

4. The Data Layer

Everything downstream is a function of this layer. It is also where the system's largest caveat lives.

4.1 Synthetic market generator

`generate_synthetic_data.py` builds five years (1,825 days, 2020-01-01 → 2024-12-29) of daily observations with four superimposed components. The whole process is seeded with `np.random.seed(42)`, so the dataset is byte-identical on every regeneration — which is what allows the forecast numbers in this document to be reproducible.

4.1.1 Mean-reverting core — Ornstein–Uhlenbeck

Freight rates are modelled as a discrete Ornstein–Uhlenbeck (OU) diffusion, the standard first-order model for a mean-reverting spread or level:

$$r(t) = r(t-1) + \theta(\mu - r(t-1)) + \sigma(t) \cdot \varepsilon(t)$$

where $\varepsilon(t) \sim \mathcal{N}(0, 1)$ and $r(t) \geq 0.2\mu$ (floor applied)

EQ 4.1

$r(t)$ freight rate, USD/tonne	μ long-run mean = lane's <code>historical_avg_rate</code>	θ mean-reversion speed = 0.03 for every lane
$\sigma(t)$ conditional volatility from the GARCH state	$\varepsilon(t)$ standard normal shock	Half-life $\ln 2 / 0.03 \approx$ 23 days

A half-life of roughly 23 days is the reason the generated series look so smooth: deviations from the mean decay fast, and the rate is pulled back toward `historical_avg_rate` almost every week. For AUS_PAR the realised five-year statistics are a mean of **\$12.96/t** inside a range of **\$12.34 – \$13.61**. Section 14 explains why this matters for model evaluation.

4.1.2 Volatility clustering — GARCH(1,1)

Conditional variance follows a GARCH(1,1) recursion, which is what produces the "calm then turbulent" texture real freight markets have and which a constant-variance simulator cannot:

$$h(t) = \alpha_0 + \alpha_1 \cdot \varepsilon_{t-1}^2 + \beta_1 \cdot h(t-1), \quad \sigma(t) = \sqrt{h(t)}$$

EQ 4.2

$\alpha_0 = 0.05 \cdot h(0)$ — intercept	$\alpha_1 = 0.10$ — ARCH reaction to last shock	$\beta_1 = 0.85$ — persistence of prior variance
$\alpha_1 + \beta_1 = 0.95 < 1 \rightarrow$ covariance-stationary		

The sum $\alpha_1 + \beta_1 = 0.95$ is the stationarity condition. At 0.95 the implied variance shock persistence half-life is ≈ 13.5 trading days, which is the correct order of magnitude for a bulk freight volatility cluster.

4.1.3 Seasonality — Fourier harmonics

Two sinusoidal harmonics are layered in as daily increments, one anchored on the origin region's demand peak and one on a secondary peak (northern-hemisphere winter, or the southern-hemisphere equivalent for Australia/Indonesia/Mozambique):

$$S(t) = [A_1 \cdot \sin(2\pi(t - 30.44(m_1 - 3))/365) + A_2 \cdot \sin(2\pi(t - 30.44(m_2 - 3))/365)] \times 0.01$$

EQ 4.3

ORIGIN REGION	Σ (DAILY VOL.)	PRIMARY PEAK	A_1	SECONDARY PEAK	A_2
Australia	0.24	October	2.0	March	0.9
US	0.28	September	2.5	March	1.2
Mozambique	0.25	November	2.2	June	1.0
Russia	0.30	December	2.8	May	1.2
Indonesia	0.22	July	1.8	December	0.8

Higher-volatility origins (Russia 0.30, US 0.28) carry both wider bands and a stronger seasonal swing — an intentional design choice, since Atlantic and Russian-origin coal routes are genuinely more volatile than Indonesian nickel/coal short-haul.

4.1.4 Co-movement with BDI and bunkers

$$r(t) = r(t-1) + \theta(\mu - r(t-1)) + S(t) + \sigma(t)\varepsilon(t) + \beta_{\text{bdi}} \cdot \Delta BDI(t) + \beta_{\text{bnr}} \cdot \Delta B(t)$$

EQ 4.4

with $\beta_{\text{bdi}} = 0.002 \cdot \mu / 1500$ (a rising dry index lifts the freight rate) and $\beta_{\text{bunker}} = -0.001 \cdot \mu / 580$ (**negative** — a higher fuel price compresses the netback the shipper can pay). That sign convention is the correct economic one and is worth calling out because it is frequently got wrong in naive simulators.

4.1.5 Market indicator series

SERIES	PROCESS	PARAMS	REALISED RANGE	ROLE
Baltic Dry Index	OU + seasonal ripple	$\mu=1500$, $\theta=0.02$, $\sigma=80$, floor 300	561 – 2,712	Global dry-bulk demand proxy; lagged 1/7/14/30 d
VLSFO bunker	OU, clipped	$\mu=580$, $\theta=0.015$, $\sigma=12$, clip [300,1000]	399 – 762	Fuel cost driver; lagged 1/7/14/30 d
Trade volume index	OU + seasonal	$\mu=100$, $\theta=0.02$, $\sigma=1.5$, clip [60,140]	60 – 140	Global trade activity level

4.2 Feature engineering — the 52 predictors

For each of the 35 lanes a feature file is built. All 52 columns listed in `FEATURE_COLS` (`models/forecasting.py:82`) fall into seven families. Formulas are given so each can be reimplemented or audited independently.

FAMILY	COUNT	COLUMNS	DEFINITION
Lagged rates	7	<code>rate_lag_1,3,7,14,30,60,90</code>	$r(t-k)$ — direct autoregressive terms
Rolling statistics	12	<code>rate_rolling_{mean,std,min,max}_{7,14,30}</code>	Sample mean / std / min / max of r over the trailing w days. Encodes the local level <i>and</i> the local range.
Returns	3	<code>rate_return_{1d,7d,30d}</code>	$r(t)/r(t-k) - 1$
Calendar	4	<code>month, quarter, day_of_week, day_of_year</code>	Raw calendar integers
Fourier	6	<code>{sin,cos}_{annual,semi_annual,quarterly}</code>	$\sin/\cos(2\pi \cdot \text{doy}/365)$, $\sin/\cos(4\pi \cdot \text{doy}/365)$, $\sin/\cos(8\pi \cdot \text{doy}/365)$. Cycles are encoded continuously so the learner never has to treat day 365 as adjacent to day 1.
Market levels	3	<code>bdi, bunker_price, trade_volume</code>	Contemporaneous indicator values
Market lags & deltas	13	<code>{bdi,bunker}_lag_{1,7,14,30}</code> , <code>{bdi,bunker}_change_{7d,30d}</code>	8 lagged levels + 4 percentage changes + <code>trade_volume</code>
Realised volatility	3	<code>realized_vol_{7,14,30}</code>	$\text{std}(\Delta r/r)_{\{w\}} \cdot \sqrt{252}$ — annualised, the market-standard convention
Momentum	2	<code>momentum_30d, momentum_90d</code>	$r(t)/MA_w(r) - 1$ — distance from the moving average

Target variable. `target_rate_1d = rate.shift(-1)` — a one-step-ahead point forecast. Two further targets (`target_rate_7d` , `target_rate_30d`) are generated and written to the CSV for future multi-horizon work, but only the 1-day target is trained on.

1,705

ROWS PER LANE AFTER NAN DROP

57

COLUMNS (52 FEATURES + DATE + RATE + 3 TARGETS)

59,675

TOTAL TRAINING ROWS, ALL 35 LANES

Row count is 1,825 minus 120: the longest lag/rolling window is 90 days and `dropna()` removes the warm-up period. Each lane is trained independently — there is no cross-lane pooling, so a lane with 1,705 samples trains a model from scratch on first request.

4.3 Reference tables

4.3.1 The 35 PS-26006 trade lanes

Five origin regions × seven East Coast discharge ports. Origin and destination jointly determine the lane code (`AUS_PAR` , `RUS_HAL` , ...) and the lane's `historical_avg_rate` is the OU process mean μ , so it feeds straight into both the generator and the forecaster's prior.

ORIGIN	PARADIP PAR	VIZAG VIZ	GANGAVARAM GAN	GOPALPUR GOP	DHAMRA DHA	SAGAR-SANDHEADS SAG	HALDIA HAL
Newcastle, Australia	13.0	13.4	13.5	13.2	12.9	14.3	14.4
US Gulf	15.0	15.4	15.5	15.2	14.9	16.3	16.4
Nacala, Mozambique	13.5	13.9	14.0	13.7	13.4	14.8	14.9
Taman, Russia	12.5	12.9	13.0	12.7	12.4	13.8	13.9
Taboneo, Indonesia	9.5	9.9	10.0	9.7	9.4	10.8	10.9

Values are `historical_avg_rate` in USD/tonne. Two consistent patterns are visible and are economically sensible: (a) rate rises with voyage distance — Sagar-Sandheads and Haldia are the longest Indian legs; (b) rate falls with distance from the origin — Indonesia at 3,200 nm is roughly one-third of the US Gulf at 10,500 nm. Voyage durations run 9–14 days for Indonesia/Australia, 19–21 for Mozambique, 21–23 for Russia, and 31–32 for the US Gulf — implying a consistent 14.1–14.6 kn service speed across every lane.

4.3.2 The 7 modelled hulls, with TPC curves

CLASS / SUBTYPE	DWT	LOA M	BEAM M	MAX DRAFT M	\$/DAY	FUEL T/DAY	TPC RANGE (T/CM)
Handysize · Standard	33,000	170.0	27.0	11.5	11,000	25	30.0 → 38.0
Supramax · Standard	58,000	190.0	32.2	13.5	13,000	30	42.0 → 52.0
Supramax · Ultramax	64,000	199.0	32.3	14.0	14,000	32	44.0 → 55.5
Panamax · Standard	75,000	229.0	32.3	14.5	15,000	35	52.0 → 62.8
Panamax · Kamsarmax	82,000	229.0	32.3	14.5	16,000	37	54.5 → 65.5
Capesize · Standard	180,000	292.0	45.0	18.2	22,000	55	78.5 → 93.0
Capesize · Newcastlemax	208,000	300.0	47.0	18.5	24,000	60	85.0 → 101.5

Each hull carries a **7-point TPC curve** — tonnes per centimetre of immersion against draft. TPC rises with draft as the waterplane area widens toward the parallel middle body. Worked example, Panamax Standard: 52.0 t/cm at 8.0 m → 62.8 t/cm at 14.5 m, a 21% rise. The 7 points are linearly interpolated in `interpolate_tpc()` and integrated in `calculate_lightering_quantity()` — see Section 8.

4.3.3 The 33 ports

Each port record carries: `max_draft_m` , `max_loa_m` , `max_beam_m` , `cargo_handling_rate_tpd` , `avg_waiting_days` , `berth_availability` , `port_charges_usd_per_tonne` , and a 12-month `weather_delay_prob` map.

PORT		DRAFT M	LOA M	BEAM M	T/DAY	WAIT D	BERTH	\$/T	WX PROB RANGE
Paradip	DISCHARGE	18.5	300	45	75,000	4.5	0.70	1.50	0.04 – 0.25
Visakhapatnam	DISCHARGE	17.1	260	40	65,000	3.5	0.74	1.60	0.04 – 0.22
Gangavaram	DISCHARGE	18.0	300	45	80,000	2.5	0.82	1.55	0.04 – 0.23
Gopalpur	DISCHARGE	13.0	230	34	45,000	3.0	0.78	1.45	0.04 – 0.23
Dhamra	DISCHARGE	18.0	300	45	75,000	2.5	0.82	1.35	0.05 – 0.25
Sagar-Sandheads	DISCHARGE	11.5	210	32	35,000	6.0	0.62	1.55	0.04 – 0.23
Haldia	DISCHARGE	10.5	230	32	40,000	5.0	0.65	1.40	0.05 – 0.28
Newcastle	LOAD	16.5	250	38	70,000	4.0	0.78	2.30	0.06 – 0.12
US Gulf	LOAD	14.0	245	34	55,000	2.5	0.85	2.80	—
Nacala	LOAD	14.5	245	34	45,000	3.5	0.76	2.10	—
Taman	LOAD	15.0	250	34	50,000	4.0	0.72	2.00	—
Taboneo	LOAD	14.0	245	34	55,000	4.0	0.72	1.80	—

Only 12 of the 33 ports appear in any PS-26006 lane — 5 load ports (Newcastle, US Gulf, Nacala, Taman, Taboneo) and 7 discharge ports. The remaining 21 are reference ballast and transhipment entries (Rotterdam, Singapore, Richards Bay, Mundra, Santos, Qingdao and others). The seven discharge ports span 8 metres of draft (10.5 → 18.5 m) and 40,000 → 80,000 t/day of handling capacity — that spread is precisely what makes the vessel optimizer non-trivial, and it is why Gopalpur, Haldia and Sagar-Sandheads force small hulls or multi-voyage plans (§6.4).

4.3.4 The monsoon signal in the weather maps

The 12-month `weather_delay_prob` map is the most operationally realistic piece of data in the repository, because it reproduces the real East Coast Indian monsoon timing:

PORT	JAN	FEB	MAR	APR	MAY	JUN	JUL	AUG	SEP	OCT	NOV	DEC
Paradip	.05	.04	.04	.06	.10	.20	.25	.22	.15	.10	.06	.05
Haldia	.06	.05	.05	.08	.12	.22	.28	.25	.18	.12	.08	.06
Newcastle (load)	.08	.07	.06	.08	.10	.12	.10	.08	.07	.06	.07	.08

Peak disruption in July–August at the discharge ports, near-flat at the load ports — correct for the Bay of Bengal branch of the South-West monsoon versus a sheltered Australian load port. This drives both the cost engine’s weather-disruption line and the lightering module’s hard weather veto.

5. Module 1 — Freight Rate Forecasting

A three-member ensemble with genuine quantile-based confidence intervals, in `backend/models/forecasting.py`.

5.1 Why an ensemble, and why this specific combination

MEMBER	ROLE	JUSTIFICATION
XGBoost reg:squarederror	Point estimate, member ½	300 trees, depth 6, lr 0.05. Captures the feature <i>interactions</i> that dominate tabular shipping data (e.g. "high BDI matters only when momentum is positive").
XGBoost reg:quantileerror	Confidence bounds ×4	Trained separately at α = 0.025, 0.10, 0.90, 0.975. Each estimates a conditional quantile of the distribution, not a symmetric error bar. This is the only component that makes the intervals honest.
LightGBM	Point estimate, member ½	Leaf-wise growth and a different binning scheme produce decorrelated errors from XGBoost, so the 50/50 average genuinely reduces variance rather than re-weighting one model.
LSTM (PyTorch)	Residual correction, h ≤ 7 d	Trained on the <i>residuals</i> of the tree ensemble, not on the raw series. A tree ensemble cannot represent smooth sequential patterns in what is left over; an LSTM can. Decay factor 1−(h−1)/7 fades it out by day 7.

$$\hat{y}(h) = 0.5 \cdot XGB_{med}(h) + 0.5 \cdot LGBM(h) + \mathbf{1}_{[h \leq 7]} \cdot LSTM(h) \cdot (1 - (h-1)/7)$$

EQ 5.1 — final point forecast

5.2 LSTM architecture

```
FreightLSTM(input_size=52, hidden_size=64, num_layers=2, dropout=0.2)
├ nn.LSTM(52 → 64, 2 layers, batch_first=True, dropout=0.2)
├ take last timestep output: lstm_out[:, -1, :]
├ nn.Dropout(0.2)
└ nn.Linear(64 → 1)           # scalar residual correction, in $/tonne
```

30

LOOKBACK WINDOW (DAYS)

50

FULL-BATCH EPOCHS

1e-3

ADAM LEARNING RATE

MSE

LOSS FUNCTION

Parameter count 63,553 (LSTM 63,488 + Linear 65), verified by `sum(p.numel() for p in model.parameters())`. Training is full-batch over 1,334 sliding sequences (1,364 − 30 lookback), 50 epochs × one step — a deliberate trade: it keeps a cold request inside the 30–60 s budget at the cost of a properly shuffled mini-batch loop. A production version should switch to `DataLoader` with batch size 64 and early stopping on a validation residual split.

5.3 Training procedure

#	STEP	DETAIL
1	Load <code>features_<ROUTE>.csv</code>	1,705 rows. Raises <code>FileNotFoundError</code> with an actionable message if the generator has not been run.
2	Chronological 80/20 split	<code>split_idx = int(0.8*1705) = 1364</code> . Train 1,364 · validation 341. No shuffling — a shuffled split would leak future information into the training set and inflate the reported error.
3	Standardise features	<code>StandardScaler</code> fitted on train only, applied to validation. Required for the LSTM; harmless and mildly helpful for the trees.
4	Fit XGBoost median	300 estimators, depth 6, lr 0.05, <code>subsample=0.8</code> , <code>colsample_bytree=0.8</code> , seed 42, with <code>eval_set</code> for early-stopping signal.
5	Fit 4 quantile XGBoosts	200 estimators, depth 5, lr 0.05. $\alpha = 0.10 / 0.90 \rightarrow 80\%$ band; $\alpha = 0.025 / 0.975 \rightarrow 95\%$ band.
6	Fit LightGBM	300 estimators, depth 6, lr 0.05, same subsampling, seed 42.
7	Compute ensemble residuals	$\varepsilon = y - (0.5 \cdot \text{XGB} + 0.5 \cdot \text{LGBM})$ on the training split.
8	Build 30-day sequences, fit LSTM	<code>_create_sequences()</code> builds overlapping windows; a second scaler standardises the LSTM inputs; Adam/MSE for 50 epochs; then <code>.eval()</code> .
9	Score on validation	MAE, RMSE, MAPE of the tree ensemble (the LSTM residual term is not scored — see §14.3).
10	Cache by route	<code>_forecaster_cache[route_code]</code> . Verified: first request for a lane 6.55 s, second 0.10 s.

5.4 Confidence interval mathematics

Interval half-widths come from the quantile models and are then **scaled by $\sqrt{h}$** for multi-step horizons:

$$W_{80}(h) = [Q_{0.90}(x_h) - Q_{0.10}(x_h)] \cdot \sqrt{h}$$
$$W_{95}(h) = [Q_{0.975}(x_h) - Q_{0.025}(x_h)] \cdot \sqrt{h}$$

EQ 5.2

The $\sqrt{h}$ factor is not arbitrary. For a random walk, the variance of an h -step forecast error is h times the one-step variance, so the standard deviation grows as $\sqrt{h}$. Applying the same reasoning to a quantile spread gives the widening above. Empirically on AUS_PAR the 95% band grows from \$13.36–\$13.45 at day 1 to \$12.09–\$14.63 at day 30 and \$11.16–\$15.55 at day 90 — a band of **0.67% $\rightarrow$ 19.0% $\rightarrow$ 32.9%** of the point forecast. The bands are correctly ordered (95% $\supset$ 80%) and correctly widening.

CAVEAT ON THE MULTI-STEP PATH

The forecast is generated by iterating the one-step model. Only the `rate_lag_1` column is refreshed between steps; the other 51 features are frozen at their last observed values. The consequence is visible in the output: day 1 = \$13.40, day 3 = \$13.36, and days 5–90 all sit at \$13.36. **The point path is effectively flat after day 3.** The confidence bands continue to widen correctly, so the interval is informative even though the central path is not. Section 14.3 records this as the highest-priority modelling fix.

5.5 Measured performance and what it means

METRIC	AUS_PAR	INTERPRETATION
MAE	\$0.0357 / t	Average absolute miss on a \$13/t rate = 0.27% of level.
RMSE	\$0.0450 / t	RMSE > MAE as expected — a small number of larger misses inflate the quadratic penalty.
MAPE	0.274 %	Reported on the dashboard's "Model MAPE" card.
Validation samples	341	20% of 1,705, chronologically the most recent 341 days.

5.5.1 Measured feature importance (XGBoost split gain, AUS_PAR, top 10)

FEATURE	FAMILY	SHARE OF TOTAL GAIN	%
rate_rolling_max_7	Rolling		42.4
rate_lag_1	Lag		39.8
rate_rolling_min_7	Rolling		8.3
rate_rolling_mean_7	Rolling		2.8
momentum_30d	Momentum		0.5
rate_return_1d	Return		0.5
rate_rolling_mean_14	Rolling		0.4
day_of_year	Calendar		0.4
rate_rolling_min_30	Rolling		0.3
rate_rolling_max_14	Rolling		0.3
All BDI, bunker, trade-volume and Fourier seasonal terms, combined			2.1

Total split gain 1.502. The top two features alone account for 82.2% of all gain in the model, and both are functions of the rate's own recent history.

THIS IS THE MOST IMPORTANT DIAGNOSTIC IN THE DOCUMENT

Gain is concentrated almost entirely in `rate_rolling_max_7` and `rate_lag_1` — that is, the learner has discovered that **tomorrow's rate $\approx$ today's rate**. Every exogenous driver, taken together, contributes 2.1%. The reason is structural, not a coding fault: the OU generator reverts to a constant μ with a 23-day half-life and a narrow realised range, so the series is close to a martingale and persistence is very nearly the optimal predictor. **The 0.274% MAPE therefore measures the generator's smoothness, not predictive skill**. On real, regime-switching freight data the importance distribution would be expected to shift materially toward BDI, bunker and the seasonal Fourier terms. This must be re-measured on approved data before any accuracy claim is made.

6. Module 2 — Vessel Optimizer

Constraint-first vessel selection and multi-voyage planning, in `backend/models/vessel_optimizer.py`. Runs in under 10 ms — it contains no model inference.

6.1 Design principle: gates, then scores

The optimizer never ranks an infeasible ship. It applies five **hard gates** that produce a rejection list with human-readable reasons, and only then scores the survivors. This is why the API can return *"Capesize (Newcastlemax) rejected: LOA 300 m exceeds route limit 250 m"* — the desk learns *why* the obvious large ship is unavailable, not merely that it lost.

6.2 The five feasibility gates

GATE	CONSTRAINT	RULE AND RATIONALE
G1	Parcel vs capacity	$Q \leq 0.95 \cdot \text{DWT}$. The 5% margin is an explicit operational reserve — the code comment records it as preventing a recommendation "at 100% DWT".
G2	Loaded draft	$d(Q) \leq \min(d_{\text{origin}}, d_{\text{dest}})$. The minimum of the two ports binds, because a ship must be able to enter both. This single gate is what excludes Capesize from most PS-26006 lanes.
G3	LOA	$\text{LOA} \leq \min(\text{LOA}_{\text{origin}}, \text{LOA}_{\text{dest}})$ — berth and approach-channel length.
G4	Beam	$B \leq \min(B_{\text{origin}}, B_{\text{dest}})$ — berth width and channel constraint.
G5	Handling rate	Not a rejection gate but a score input: $\text{handling_days} = Q / \min(\text{tpd}_{\text{origin}}, \text{tpd}_{\text{dest}})$. The slower berth sets the pace.

6.2.1 Draft estimation model

Draft is interpolated linearly between the vessel's light draft (first TPC-curve point) and its maximum designed draft, scaled by the load factor:

$$\lambda = \min(1, Q / \text{DWT}) \quad d(Q) = d_{\min} + (d_{\max} - d_{\min}) \cdot \lambda$$

EQ 6.1

A real hydrostatic calculation would integrate the vessel's actual waterplane area against cargo weight. This linear surrogate is monotone, correctly bounded, and cheap — adequate for a feasibility screen, and the reason the same function can be reused by the lightering module without inconsistency.

6.2.2 Material preference matrix

Material is a **preference, not an exclusion** — the code states this explicitly. Feasibility gates decide admissibility; material fit only shifts the score.

MATERIAL	HANDYSIZE	SUPRAMAX	PANAMAX	CAPE SIZE	RATIONALE
Coal	0.65	0.90	1.00	0.95	Panamax is the global coal standard; large coal parcels want big holds.
Steel	1.00	0.95	0.82	0.65	Steel is dense and parcelised — smaller holds suit stowage and grab handling.

Consequence worth noting: for the *same* 55,000 t parcel on Newcastle→Paradip, coal selects **Panamax (Standard)** and steel selects **Supramax (Ultramax)** — the material genuinely changes the ship.

6.3 The feasibility score

$$S = 100 \cdot [0.40 \cdot M + 0.30 \cdot U + 0.20 \cdot T + 0.10 \cdot 1 [d \leq d_{\text{lim}}]]$$

EQ 6.2

$$U = \max(0, 1 - |\lambda - 0.72|) \leftarrow \text{utilisation, peaked at 72\% DWT}$$

$$T = \max(0, 1 - \min(1, (H + W)/25)) \leftarrow \text{turnaround}$$

$$W = w_o/b_o + w_d/b_d \leftarrow \text{expected waiting days}$$

$$H = Q / \min(\text{tpd}_o, \text{tpd}_d) \leftarrow \text{cargo handling days}$$

0.40 · M material fit — dominant weight

0.30 · U utilisation, ideal 72% DWT

0.20 · T turnaround exposure

0.10 · 1 draft compliance (1 after gates pass)

The waiting model W is the port's mean wait deflated by berth availability. For Newcastle→Paradip: $4.0/0.78 + 4.5/0.70 = 5.13 + 6.43 = 11.56$ d. This is the single largest non-freight cost in the system and is revisited in §7.4.

Tie-breaking. Candidates sort by `(feasibility_score, -class_order)` descending, so equal scores resolve to the *larger* class — a deliberate preference for fewer voyages at equal fit.

6.4 Multi-voyage planning

When no single hull clears gate G1, the optimizer does not give up. It computes the largest parcel each hull can carry *within the route draft limit*, then solves for the voyage count:

$$f_d = \min(1, (d_{\text{lim}} - d_{\text{min}})/(d_{\text{max}} - d_{\text{min}}))$$

$$C_{\text{voy}} = \min(0.95 \cdot \text{DWT}, \text{DWT} \cdot f_d)$$

$$n = \lceil Q / C_{\text{voy}} \rceil \quad \text{per-voyage parcel} = Q/n$$

EQ 6.3

The split-mode score re-weights the objective to suit the new problem:

$$S_{\text{split}} = 100 \cdot [0.45 \cdot (1/n) + 0.30 \cdot M + 0.15 \cdot U + 0.10 \cdot T]$$

EQ 6.4

Minimising voyage count is now the primary term at 45% weight — a two-voyage plan on bigger ships beats a five-voyage plan on Handysizes even at worse material fit. Candidates sort by `(split_voyages, -feasibility_score)`: fewest voyages first.

6.5 Status codes

STATUS	MEANING	DOWNSTREAM BEHAVIOUR
FEASIBLE	One hull carries the whole parcel.	Cost, lightering and decision modules run on that hull at full parcel.
FEASIBLE_SPLIT	Parcel requires n voyages.	Modules run on <code>per_voyage_cargo_tonnes</code> ; the cost total is multiplied by n and cost/tonne recomputed on the original parcel.
NO_FEASIBLE_VESSEL	No hull in the fleet satisfies the constraints, even split.	Cost, lightering and decision modules return <code>null</code> ; the frontend shows an advisory to change port or parcel.

6.6 Verified behaviour — six parcels, six outcomes

LANE · PARCEL	STATUS	SELECTED	SCORE	CONSTRAINT THAT DOMINATED
AUS_PAR · 55,000 t coal	FEASIBLE	Panamax (Standard)	89.7	Capesize blocked by Newcastle LOA 250 m / beam 38 m; Handysize too small
AUS_PAR · 55,000 t steel	FEASIBLE	Supramax (Ultramax)	83.9	Material fit inverts the choice vs. the coal case
AUS_PAR · 120,000 t coal	SPLIT	Panamax (Kamsarmax) ×2 @ 60 kt	72.4	All 7 hulls failed G1; 2 voyages minimise cost
AUS_HAL · 120,000 t coal	SPLIT	Handysize ×5 @ 24 kt	48.0	Haldia draft 10.5 m — every hull ≥ Supramax fails G2 and G3
USA_GOP · 40,000 t coal	FEASIBLE	Supramax (Standard)	88.9	Gopalpur LOA 230 m / beam 34 m exclude both Capesizes
RUS_SAG · 150,000 t coal	SPLIT	Handysize ×5 @ 30 kt	44.2	Sagar-Sandheads draft 11.5 m, LOA 210 m — 4 distinct gates fire per hull

The Haldia and Sagar-Sandheads rows are the payoff of the constraint-first design: a 120,000 t and a 150,000 t parcel respectively reduce to 5 Handysize voyages because the discharge berth physically cannot take anything larger. A price-scoring approach would have "recommended" a Capesize and been wrong.

7. Module 3 — Total Delivered Cost Engine

Eight additive cost components, in `backend/models/cost_calculator.py`. The point of the module is the components the freight quote does not contain.

7.1 The cost identity

$$C_{\text{total}} = C_{\text{freight}} + C_{\text{port}} + C_{\text{wait}} + C_{\text{demurrage}} + C_{\text{disrupt}} + C_{\text{canal}} + C_{\text{ins}} + C_{\text{bunker}}$$
$$C_{\text{per tonne}} = C_{\text{total}} / Q$$

EQ 7.1

NOTE ON THE "NINE COMPONENTS" CLAIM

The README advertises nine cost components. The summation in `calculate_total_cost()` contains **eight**. The ninth — lightering — is computed by a separate module and rendered on its own tab, but it is **not** added to `total_cost_usd`. So on a lightering voyage the delivered-cost total understates the true cost by the entire STS operation. This is a documentation/code mismatch and a real accounting gap; both are logged in Section 14.

7.2 Component-by-component specification

COMPONENT	FORMULA AS IMPLEMENTED	NOTES
1 • Freight	$C = r \cdot Q$	r = forecaster day-1 rate (USD/t). The only component quoted by a broker.
2 • Port charges	$C = (p_o + p_d) \cdot Q$	Per-tonne tariff at each end. For Newcastle (\$2.30) → Paradip (\$1.50) the \$0.80/t spread is 6.0% of the freight rate — and the load port is <i>more</i> expensive here, which inverts the usual intuition.
3 • Waiting time	$W = w_o/b_o + w_d/b_d$ $C = W \cdot \text{daily_rate}$	Expected berth wait at both ends, charged at the vessel's daily charter rate. See §7.4 — this is the module's largest line item after freight.
4 • Demurrage	$\text{rate} = 1.5 \cdot \text{daily_rate}$ $\text{days} = \max(0, 2Q/(\text{DWT}/3.5) - 2 \cdot L)$ $C = \text{days} \cdot \text{rate}$	Loading rate assumed DWT/3.5 t/day; port time doubled for load + discharge; $L = 5$ allowed laydays per call. The 1.5× multiplier on the daily rate is the standard charter-party convention.
5 • Weather disruption	$C = (p_o(m) + p_d(m)) \cdot 3.0 \cdot \text{daily_rate}$	Monthly weather-delay probabilities from <code>ports.json</code> for the chosen loading month; 3.0 days is the assumed mean event duration. Directly month-sensitive — see §4.3.4.
6 • Canal transit	Suez: $\text{SCNT} \cdot \text{rate}$, $\text{SCNT} = 0.6 \cdot \text{DWT}$ Panama: $\text{PCUMS} \cdot \text{rate}$, $\text{PCUMS} = 0.55 \cdot \text{DWT}$	Tiered tariffs (Suez \$7.88 down to \$4.53/SCNT; Panama \$5.25 → \$5.01/PCUMS). Currently unreachable — see §14.2.
7 • Insurance	$C = 0.002 \cdot v \cdot Q$	Marine cargo at 0.2% of cargo value, inside the 0.1–0.3% market range.
8 • Bunker	$C = \text{fuel_t_day} \cdot D \cdot \text{bunker_price}$	Fuel consumption × route voyage days × VLSFO price. The single largest exposure to the bunker input: a \$50/t move changes cost by \$22,200 on a 12-day Panamax voyage.

7.3 Worked cost breakdown — AUS_PAR, 120,000 t coal, June

As returned by `POST /api/full-analysis`. Two Panamax (Kamsarmax) voyages of 60,000 t each; freight rate = forecaster day-1 value \$13.40/t; cargo value \$120/t; VLSFO \$580/t.

COMPONENT	PER VOYAGE (USD)	× 2 VOYAGES	SHARE	DRIVER
Freight	804,000.00	1,608,000.00	53.5%	13.40 × 60,000
Port charges — loading	138,000.00	276,000.00	9.2%	Newcastle \$2.30/t
Port charges — discharge	90,000.00	180,000.00	6.0%	Paradip \$1.50/t
Waiting time 11.6 D	184,908.42	369,816.84	12.3%	$(4.0/0.78 + 4.5/0.70) \times \$16,000$
Demurrage 0.0 D	0.00	0.00	0.0%	10.2 d port time < 10 d allowed
Weather disruption 1.0 D	15,360.00	30,720.00	1.0%	$(0.12 + 0.20) \times 3 \text{ d} \times \$16,000$
Canal fees	0.00	0.00	0.0%	Lane has no canal transit
Insurance	14,400.00	28,800.00	1.0%	$0.2\% \times \$120 \times 120,000$
Bunker 444 T	257,520.00	515,040.00	17.1%	37 t/day × 12 d × \$580
TOTAL	1,504,188.42	3,008,376.84	100%	\$25.07 / tonne delivered

Freight is 53.5% of delivered cost — **46.5% of the money is invisible in the broker's quote**. The two largest hidden lines are bunker (17.1%) and waiting time (12.3%). Together they are \$884,857, or \$7.37/tonne, on a voyage whose freight line is \$13.40/tonne. This is the single strongest quantitative argument for the platform: optimising on freight alone optimises barely half the decision.

7.4 Sensitivity — cost per tonne against vessel size

Same lane, same rate, same month; only the parcel and hull change. This is the comparison `compare_vessel_options()` surfaces on the Cost Analysis tab.

PARCEL	HULL	TOTAL USD	\$/TONNE	VS. CAPE SIZE	WHY
33,000 t	Handysize	887,205	26.88	+27.4%	Highest \$/dwt — bunker and waiting are near-fixed, so a small parcel amortises them badly.
58,000 t	Supramax	1,383,038	23.85	+13.0%	Better amortisation, but 2 voyages needed for the 120 kt contract.
75,000 t	Panamax	1,739,352	23.19	+9.9%	The largest hull permitted by Newcastle (LOA 250 m, beam 38 m).
180,000 t	Capesize	3,797,369	21.10	—	Best \$/t, but physically excluded by Newcastle's LOA and beam limits.

THE ECONOMICS AND THE PHYSICS DISAGREE — BY DESIGN

The cheapest vessel per tonne on this lane is the Capesize, and it is **not an option**. The optimizer returns the Panamax at \$25.07/t, 18.8% above the unconstrained optimum. This is the correct behaviour of a constraint-first system and it quantifies, in dollars, exactly what Newcastle's berth geometry costs SAIL on Australian coal. Presenting the Capesize number as "the alternative" would be a modelling error; the platform only ever shows hulls that passed the gates.

7.5 Known weaknesses in the cost model

- **Waiting cost is charged on both ports for the full voyage.** It is an expected-value line, not a schedule simulation, and it treats a load-side wait and a discharge-side wait identically. At 12.3% of total it deserves a proper queueing or agent-based treatment before production use.
- **Demurrage is a deterministic threshold**, not a distribution. Real demurrage is a function of berth-window variance, which the waiting model already hints at but does not couple.
- **Insurance is a flat 0.2%** with no war-risk, sanctions or lay-up loading.

- **No cargo-handling cost** (stevedoring, tally, shunting) — usually borne by the charterer and often larger than insurance.
- **No ballast or repositioning cost**, which matters directly for the idle-management question in §9.6.
- **No time-value treatment inside the voyage**; everything is undiscounted and evaluated at fixture date.

8. Module 4 — Lightering Optimizer

Ship-to-ship cargo transfer solved from the vessel's TPC curve, in `backend/models/lightering.py`.

8.1 The naval-architecture problem

A loaded ship draws more water than an empty one. If her loaded draft exceeds the discharge berth's charted depth, she must transfer cargo into a lighter *at anchor* until she draws less. The quantity to transfer is not proportional to the draft excess, because the ship becomes more buoyant-per-centimetre as she rises toward the waterline. That non-linearity is exactly what the TPC curve encodes.

TPC — TONNES PER CENTIMETRE

The immersion (weight) required to sink a vessel one further centimetre, as a function of her current draft. It is the first derivative of displacement with respect to draft — the discrete form of the waterplane area $A(d)$. Because $A(d)$ widens as a hull approaches its parallel middle body, TPC rises with draft. In this dataset TPC rises about 20% from light to fully loaded on every hull — which is exactly why a linear draft-to-tonnage conversion would under-transfer.

8.2 The integration

$$d_{\text{target}} = d_{\text{port}} - m_{\text{UKC}} \leftarrow 0.5 \text{ m under-keel clearance by default}$$

EQ 8.1

$$W = \int_{d_{\text{target}}}^{d_{\text{loaded}}} \text{TPC}(d) \, dd \cdot 100$$

The $\times 100$ converts metres to centimetres. The integral is evaluated as a **left-endpoint Riemann sum at 1 cm resolution**:

$$W = \sum_{k=0} \dots n-1 \text{TPC}(d_{\text{target}} + k/100) \text{ , } n = \lfloor 100 \cdot (d_{\text{loaded}} - d_{\text{target}}) \rfloor$$

EQ 8.2

At 1 cm resolution the discretisation error against a true integral of a piecewise-linear TPC curve is bounded by the TPC slope over one step — under 0.1 t on these hulls. The code additionally reports `avg_tpc` = W/n , which is the quantity a naval architect would sanity-check first, and returns the first 10 TPC samples as a `tpc_profile` for plotting.

8.2.1 Worked example 1 — 60,000 t Panamax Standard into Haldia (July)

Estimated loaded draft	13.20 m (8.0 + 6.5 × 0.800)
Haldia charted draft	10.50 m
Target draft (10.5 – 0.5 UKC)	10.00 m
Draft excess	3.20 m = 319 cm
Average TPC over range	58.5 t/cm (52.0 → 62.8 t/cm on the curve)
Cargo to lighter	18,655 t = 31% of the parcel

8.3 Lightering cost model

$$C_{\text{lig}} = W \cdot \rho_f \cdot d_{\text{op}} + C_{\text{fix}} + W \cdot \lambda \cdot v + W \cdot v \cdot 0.001$$

EQ 8.3

TERM	DEFAULT	MEANING
Feeder vessel hire	\$12,000/day × 2 d	Charter of the receiving lighter. A fixed cost regardless of tonnage moved.
STS fixed cost	\$50,000	Tugs, fenders, mooring masters, gangway, transfer hoses, permits.
Cargo loss exposure	0.2% of value	Spillage, heel, contamination. Industry range 0.1–0.3%.
Additional STS insurance	0.1% of value	Elevated-risk cover for transfer operations.

Worked example 1 continued: $24,000 + 50,000 + 4,477 + 2,239 = \$80,716$, i.e. \$4.33 per tonne transferred. The structure matters more than the total — the two fixed terms are 92% of the cost, so **STS is a step function, not a linear one**. A marginally infeasible parcel can often be solved by shaving a few hundred tonnes off the load rather than paying for a transfer. That insight is only visible because the cost is decomposed.

8.4 Weather gating — the hard veto

STS transfer is suspended when significant wave height exceeds **Hs > 1.5 m**, the OCIMF working threshold. The module converts each port's monthly delay probability into an operational risk class:

WEATHER DELAY PROB.	RISK LEVEL	OPERATIONAL GUIDANCE
< 0.08	LOW	Favourable window; STS can proceed.
0.08 – 0.15	MODERATE	Plan for 1–2 day delays.
0.15 – 0.22	HIGH	Consider alternative timing or direct berthing.
≥ 0.22	VERY HIGH	Monsoon/storm season — STS highly risky ; strongly recommend an alternative approach.

Window availability is estimated as $30 - (p \cdot 30 / 3) \cdot 3 = 30 - 30p$ usable days per month, with a 3-day assumed mean event duration. Haldia in July gives $p = 0.28 \rightarrow 21.6$ available days and 2.8 expected events.

THE VETO IS ABSOLUTE

When risk class is **VERY HIGH** the module **overrides the economic comparison entirely** and forces **REDUCE_CARGO** , regardless of how much cheaper STS happens to be. In worked example 1 the economics favoured lightering by \$12,559 (STS \$80,716 vs. \$93,275 of forgone cargo), but the July monsoon veto reversed the verdict. This is the right precedence — weather safety outranks \$12.5k — and it is the clearest example in the system of a non-financial constraint correctly dominating an economic one.

8.5 Worked example 2 — 150,000 t Capesize into Gopalpur (June)

Loaded draft → target (13.0 – 0.5)	16.83 m → 12.50 m (excess 4.33 m, 433 cm)
Average TPC	87.1 t/cm
Cargo to lighter	37,723 t = 25% of the parcel
Lightering cost	$\$87,580 \cdot \text{foregone cargo at } \$5/\text{t} = \mathbf{\$188,615}$
Verdict	LIGHTER “Lightering is \$101,035 cheaper than reducing cargo.”

At 433 cm of draft reduction the fixed STS costs amortise over far more tonnage, so the economics flip decisively in favour of transferring. The decision boundary is therefore a function of draft excess, not of the port — which is the correct behaviour and emerges from the cost structure rather than being hard-coded.

8.6 When the lightering module actually fires

An exhaustive sweep of all 35 lanes × 2 materials × 8 parcel sizes (560 feasible combinations) through the real optimizer → lightering chain gives:

DISCHARGE PORT	CHARTED DRAFT	LIGHTERING TRIGGERED	EXPLANATION
Gopalpur	13.0 m	40 / 80	The optimizer permits drafts up to 13.0 m; the 0.5 m UKC margin then forces a small transfer of ~1,570 t on Panamax loads.
Haldia	10.5 m	20 / 80	Same mechanism at a shallower berth; the optimizer has already shrunk the hull to Handysize, so the residual excess is small.
Sagar-Sandheads	11.5 m	0 / 80	Shallow, but the Handysize it forces draws ~10.0 m, inside the 11.0 m target.
Paradip · Vizag · Gangavaram · Dhamra	17.1 – 18.5 m	0 / 320	Deep enough that no load in the feasible set comes near the limit.
Total	—	60 / 560 (10.7%)	Concentrated entirely at the two shallowest berths.

This is a good result to be able to state precisely: the module is not dead code, and it is not firing spuriously. It fires exactly where physics says it must — at shallow discharge berths — and the trigger condition is the under-keel-clearance margin, not a stub.

9. Module 5 — Charter vs Spot Decision Engine

NPV comparison, analytic breakeven, and 1,000-path Monte Carlo with Value-at-Risk, in `backend/models/decision_engine.py`.

9.1 NPV of the two cash-flow streams

$$\text{NPV}(X) = \sum_{t=1}^T \text{CF}_t / (1 + r/m)^t$$

EQ 9.1

CF_t = cost paid in period t

r = 8% p.a. default discount rate

m = 12 periods/year (monthly)

T = contract months (1–24)

Charter CF = $R \cdot Q$, constant

Spot CF = $S_t \cdot Q$, S_t = forecast

Daily forecasts are aggregated into contract months in 30-day blocks, averaging both the point forecast and the two CI bounds. If `contract_months` exceeds the 90-day forecast horizon, remaining months are extrapolated flat from the last value with the band widened 15% per extra month.

9.2 Breakeven charter rate — closed form

Setting the two NPVs equal and cancelling the parcel size Q , which appears on both sides:

$$\sum R \cdot Q / (1+r)^t = \sum S_t \cdot Q / (1+r)^t$$

EQ 9.2

$$\Rightarrow R^* = \sum (S_t / (1+r)^t) \div \sum (1 / (1+r)^t)$$

So the breakeven rate is the **discount-factor-weighted average** of the forecast spot rates. Its practical value to the desk is that it is a negotiating number: "we will not sign above \$13.36/t", derived from the model rather than asserted.

If the user supplies no proposed rate, the engine defaults to $0.95 \times R^*$ — a deliberately charter-favourable stance, on the reasoning that a recommendation engine should model the case for the decision it is being asked to evaluate rather than assume a null result.

9.3 Monte Carlo design

STEP	OPERATION	DETAIL AND RATIONALE
1	Derive per-period σ from the 95% band	$\sigma_t = (U_{95} - L_{95}) / 3.92$, floored at $0.01 \cdot \text{mean}$. The 3.92 divisor is the two-sided 95% normal quantile (1.96×2), so the 95% interval is reinterpreted as $\pm 1.96\sigma$. The 1% floor prevents a degenerate zero-variance month.
2	Sample 1,000 independent paths	Each path draws T independent normals. No path dependence — periods are not correlated across time. This understates true serial correlation and is logged in §14.3.
3	Clip extremes	Rate bounded to $[0.3 \cdot \text{mean}, 2.5 \cdot \text{mean}]$. Prevents a single Gaussian tail from producing a negative or absurd freight rate.
4	Compute the savings distribution	$\text{savings} = \text{NPV}(\text{spot path}) - \text{NPV}(\text{charter})$. Positive $\Rightarrow$ charter is cheaper.
5	Probability of charter winning	$P = \text{mean}(\text{savings} > 0)$ — the empirical CDF at zero. No distributional assumption needed beyond the per-period normal.
6	Value at Risk, 95%	$\text{VaR}_{95} = \text{percentile}(\text{savings}, 5)$. Negative $\Rightarrow$ in the worst 5% of outcomes, locking the charter rate loses money.
7	Maximum potential loss	$\text{min}(\text{savings})$ — the absolute worst simulated path.
8	Histogram for the UI	30 bins over total spot cost, rendered as the Monte Carlo distribution chart.

9.4 Decision rule

$P(\text{CHARTER CHEAPER})$	VERDICT	CONFIDENCE	ACTION IMPLIED
≥ 0.85	CHARTER	HIGH	Sign. Lock the rate.
$0.70 - 0.85$	CHARTER	MODERATE	Sign, but leave a review clause.
$0.45 - 0.70$	NEUTRAL	LOW	No edge. Negotiate below breakeven or wait for clarity.
$0.25 - 0.45$	SPOT	MODERATE	Stay flexible.
≤ 0.25	SPOT	HIGH	Do not charter at this rate.

9.5 Verified worked example — AUS_PAR, 60,000 t/voyage, 6 months

QUANTITY	VALUE	READING
Breakeven charter rate R^*	\$13.36 / t	Discount-weighted mean of the monthly forecast.
Proposed rate (auto = $0.95 R^*$)	\$12.69 / t	5% inside breakeven — the charter-favourable default.
Charter NPV	\$4,464,488	$6 \times \$12.69 \times 60,000$, discounted monthly.
Spot NPV (mean of 1,000 paths)	\$4,699,965	$\pm \$152,070$ (1σ).
NPV advantage of charter	\$235,478	Spot NPV – charter NPV.
$P(\text{charter cheaper})$	93.3%	→ CHARTER · HIGH
Expected savings	\$235,478	Mean of the savings distribution.
VaR 95%	–\$27,058	In the worst 5% of paths, the charter costs \$27k more than spot.
Maximum potential loss	–\$368,654	Worst single simulated path.

The Monte Carlo distribution is tight: charter total \$4.57M against a spot mean of \$4.81M with a 1σ spread of ~\$0.15M. The tail is thin, which is why VaR is only –\$27k even though the worst observed path is –\$369k. Note the asymmetry — expected savings are 8.7× the VaR. That ratio, not the point estimate, is the number a risk committee should be shown.

A STRUCTURAL ARTEFACT IN THE CURRENT VERDICT

The 93.3% probability of charter winning is **not** a market insight. It is close to the arithmetic consequence of three design choices interacting: (i) the auto-proposed rate is set 5% below breakeven; (ii) the forecast path is nearly flat (all six monthly rates = \$13.36), so the simulated distribution is tight around a known number; (iii) σ is derived from the 95% band, which the $\sqrt{h}$ widening keeps moderate. A flat forecast plus a rate 5% below its own mean will win roughly 90–95% of the time almost regardless of the market. The verdict is arithmetically correct given the inputs and should be reported as such. On a real, volatile rate series the probability would be materially lower and far more informative. Logged as §14.3.

9.6 Operational insights — timing and idle management

Beyond the charter/spot binary, the engine derives two operational signals directly from the forecast. These address the idle-time question in the problem statement.

9.6.1 Market entry timing

A 7-day rolling mean is computed across the whole forecast; its 25th and 75th percentiles become the low and high thresholds. The current 7-day mean is then classified:

CONDITION	SIGNAL	GUIDANCE EMITTED
$current \leq Q_{25}$	FAVORABLE_WINDOW	"Review short-term charter offers now."
$current \geq Q_{75}$	WAIT_FOR_WINDOW	"For flexible cargo, monitor for a lower-rate window."
otherwise	MONITOR	"Compare charter quotes with the rolling forecast before fixing."

The module also reports the **best 7-day window** — the position of the rolling-mean minimum and its average rate — so the desk receives an actionable date range rather than a general caution.

9.6.2 Idle-scenario management

Contiguous runs of the rolling mean at or below Q_{25} are extracted as **low-demand windows**, the three cheapest retained. A low rate is used as a proxy for softer freight demand — reasonable in a market where weak rates and thin cargo coexist, though it is a proxy and not an observed cargo-volume signal.

The emitted strategy has three concrete parts: **(a)** avoid unnecessary ballast repositioning during soft windows; **(b)** keep the vessel near the relevant loading region where commercially practical; **(c)** evaluate an alternate employment on another modelled East Coast lane before committing to a long idle spell. Part (c) is the one that requires data the platform does not yet have — see §14.5.

10. Module 6 — Risk Summary

A weighted composite of three risk dimensions, in `backend/main.py :: _calculate_risk_summary()` . This is the only module with no dedicated file.

10.1 The three dimensions and their scaling

DIMENSION	SCALING TO 0–100	SOURCE
Weather risk	$\min(100, p_{wx} \cdot 500)$	Discharge-port weather-delay probability for the loading month, via the lightering module. A 0.20 probability saturates the scale.
Port congestion	$\min(100, W \cdot 10)$	Total expected waiting days from the cost engine. 10 days saturates.
Rate volatility	$\min(100, (\tilde{W}_{95}/\bar{r}) \cdot 100 \cdot 2)$	Mean 95% band width ÷ mean rate, over the first 30 forecast days. A 50% relative band saturates.

$$S_{\text{overall}} = \lfloor 0.35 \cdot S_{\text{wx}} + 0.25 \cdot S_{\text{cong}} + 0.40 \cdot S_{\text{vol}} \rfloor$$

EQ 10.1

Rates carry the highest weight (0.40) because rate risk is the one a charter decision actually transfers to the counterparty; weather (0.35) is next because it gates both cost and STS feasibility; congestion (0.25) is lowest as it is largely within the operator's own scheduling control.

Classification: < 30 LOW · 30–54 MODERATE · 55–74 HIGH · ≥ 75 VERY HIGH

10.2 Verified output — AUS_PAR, June

DIMENSION	SCORE	BAR	DERIVATION
Rate volatility	25		Day-30 95% band \$12.09–\$14.63 (width \$2.54) on a \$13.36 rate → 19.0% → ×2 = 38, averaged over 30 days ≈ 25.
Weather risk	50		Default 50. Not overwritten, because with no lightering required the module returns <code>weather_risk: null</code> . See §14.2.
Overall	52		$\lfloor 0.35 \cdot 50 + 0.25 \cdot 100 + 0.40 \cdot 25 \rfloor = \lfloor 52.5 \rfloor = 52 \rightarrow$ MODERATE
Port congestion	100		11.6 expected waiting days × 10 → saturated at 100. The binding risk on this lane is berth congestion, not the market.

The 100/100 congestion score is a genuine and actionable finding: on Newcastle→Paradip the model says the dominant risk is 11.6 days of berth waiting, worth \$7.37/tonne, and that neither the rate forecast nor the weather matters nearly as much. That is precisely the kind of insight a freight-quote-only process cannot surface, and it is a good illustration of the platform earning its place.

11. API Layer

Ten endpoints on FastAPI, all defined in `backend/main.py`.

11.1 Endpoint inventory

METHOD & PATH	REQUEST MODEL	PURPOSE
<code>GET /api/health</code>	—	Status, UTC timestamp, and <code>data_generated</code> — the flag the frontend polls to distinguish "Engine Ready" from "Generate Data First".
<code>GET /api/routes</code>	—	All 35 lane records. Drives the origin/destination dropdowns.
<code>GET /api/vessels</code>	—	All 7 hull records including TPC curves.
<code>GET /api/ports</code>	—	All 33 port records including 12-month weather maps.
<code>POST /api/recommend-vessel</code>	<code>VesselRecommendationRequest</code>	Vessel optimizer only. Returns status, recommended, alternatives, and the full rejection list.
<code>POST /api/predict-rates</code>	<code>ForecastRequest</code>	90-day forecast with 80%/95% bands, 90-day history, training metrics.
<code>POST /api/calculate-cost</code>	<code>CostRequest</code>	Single-voyage delivered cost. Vessel chosen by the caller.
<code>POST /api/lightering-analysis</code>	<code>LighteringRequest</code>	TPC integration, STS cost, weather risk, recommendation.
<code>POST /api/charter-decision</code>	<code>CharterDecisionRequest</code>	NPV + breakeven + Monte Carlo + operational insights.
<code>POST /api/full-analysis</code>	<code>FullAnalysisRequest</code>	Primary endpoint. All six modules, orchestrated, in one response.

11.2 Pydantic request schemas and validation

Validation happens at the boundary, so an out-of-range request never reaches a model. Constraints are declarative rather than hand-written checks.

SCHEMA	FIELDS WITH CONSTRAINTS
<code>ForecastRequest</code>	<code>route_code: str (required) · horizon_days: int = 30, ge=1, le=90</code>
<code>CostRequest</code>	<code>cargo_quantity_tonnes: float, gt=0 · route_code: str · vessel_class: str · freight_rate: float, gt=0 · loading_date_month: int, 1-12 · cargo_value_per_tonne: float, gt=0 · bunker_price_per_tonne: float, gt=0 · vessel_subtype: str = "Standard"</code>
<code>LighteringRequest</code>	<code>cargo_quantity_tonnes: float, gt=0 · vessel_class: str · port_name: str · month: int, 1-12 · cargo_value_per_tonne: float, gt=0 · vessel_subtype: str</code>
<code>CharterDecisionRequest</code>	<code>route_code: str · cargo_quantity_tonnes: float, gt=0 · contract_months: int = 6, ge=1, le=24 · proposed_charter_rate: Optional[float] · discount_rate: float = 0.08, gt=0, lt=1</code>
<code>FullAnalysisRequest</code>	<code>cargo_quantity_tonnes: float, gt=0 · cargo_material: str = "Coal" · route_code: str · contract_months: int, 1-24 · loading_date_month: int, 1-12 · cargo_value_per_tonne: float, gt=0 · bunker_price_per_tonne: float, gt=0 · proposed_charter_rate: Optional[float]</code>

11.3 The `full-analysis` response contract

Seven top-level keys. Verified present and populated for the reference parcel:

KEY	TYPE	CONTENTS
<code>vessel_recommendation</code>	object	<code>status</code> , <code>route</code> , <code>port_constraints</code> (both ends), <code>recommended</code> , <code>alternatives[4]</code> , <code>rejected[]</code> , <code>explanation</code>
<code>forecast</code>	object	<code>forecasts[90]</code> , <code>ci_80{lower,upper}</code> , <code>ci_95{lower,upper}</code> , <code>historical_rates[90]</code> , <code>training_metrics</code> , <code>model_info</code>
<code>cost_analysis</code>	object	<code>total_cost_usd</code> , <code>cost_per_tonne_usd</code> , <code>breakdown{8 components}</code> , <code>route_info</code> , <code>vessel_info</code> , <code>voyage_plan</code> (when split)
<code>vessel_comparison</code>	object	<code>options[]</code> priced for feasible hulls only, <code>recommended</code> , <code>selection_basis</code>
<code>lightering</code>	object	<code>vessel</code> , <code>port</code> , <code>lightering_calculation</code> , <code>lightering_cost</code> , <code>direct_berthing_cost</code> , <code>weather_risk</code> , <code>recommendation</code> , <code>reason</code>
<code>charter_decision</code>	object	<code>decision</code> , <code>confidence</code> , <code>reasoning</code> , <code>breakeven_rate</code> , <code>npv_comparison</code> , <code>monte_carlo{+histogram}</code> , <code>monthly_forecast</code> , <code>risk_metrics</code> , <code>operational_insights</code>
<code>risk_summary</code>	object	<code>overall_score</code> , <code>overall_level</code> , <code>overall_color</code> , <code>dimensions{weather, congestion, volatility}</code>
<code>warnings</code>	array	Absent when empty. Populated with one message per module that raised; the request still returns 200.

11.4 Cross-origin and error handling

CORS

Explicit allow-list of `localhost:5173`, `localhost:3000` and `127.0.0.1:5173` with credentials enabled, all methods, all headers. In development the Vite proxy (`/api` → `:8000`) makes CORS moot; it matters for a deployed build.

Error taxonomy

`ValueError` → **400** with the message (e.g. "Route not found: XXX"). `FileNotFoundError` → **400** with the instruction to run the generator. Any other exception → **500** with the message; the top-level handler also appends a full traceback. Inside `full_analysis` a module failure degrades that section to `null` and records a warning rather than aborting.

Measured latency: **6.55 s** for the first request on a cold lane (XGBoost x5, LightGBM and 50 LSTM epochs all train inside the call) and **0.10 s** for every subsequent request on the same lane. The frontend Axios timeout is set to 120 s to cover the cold case.

12. Frontend Dashboard

React 18 single-page dashboard — one form, one button, four result tabs.

12.1 Components

COMPONENT	LINES	RESPONSIBILITY
<code>App.jsx</code>	101	Header, trade-lane counter, live backend status pill, offline and no-data banners, footer.
<code>api.js</code>	33	Axios instance, <code>/api</code> base, 120 s timeout, response interceptor that unwraps FastAPI's <code>detail</code> field into a readable <code>Error</code> .
<code>Dashboard.jsx</code>	333	The 8-field input form, dependent origin→destination cascade, route-code resolution, submit orchestration, tab state, and the warning banner.
<code>VesselRecommendation.jsx</code>	80	Selected hull banner with explanation, multi-voyage notice, and — notably — the full rejection list with per-hull reasons .
<code>RiskIndicators.jsx</code>	71	Overall risk chip plus three labelled progress bars, colour-coded green → yellow → orange → red.
<code>ForecastChart.jsx</code>	215	Composed chart: 60 days history as a solid line, forecast as a dashed line, 80% and 95% bands as stacked <code>Area</code> fills. Four KPI cards and a model-details strip.
<code>CostBreakdown.jsx</code>	183	Four KPI cards, an 8-slice donut, a detailed line-item table, and a stacked bar comparing cost/tonne across feasible hulls.
<code>LighteringAnalysis.jsx</code>	197	Recommendation banner, draft/TPC/tonnage cards, the TPC profile line chart, and an option-comparison bar (lighter vs. reduce cargo).
<code>DecisionPanel.jsx</code>	252	Verdict banner, five KPI cards, market-timing and idle-scenario cards, NPV bar, 30-bin Monte Carlo histogram, and a month-by-month forecast-vs-charter table.

12.2 Input form and route resolution

Eight inputs, all in one grid: **cargo quantity**, **cargo material** (Coal / Steel), **origin region** (5), **East Coast discharge port** (7, filtered by origin), **contract period** (1–24 months), **loading month** (1–12), **cargo value \$/t**, **bunker price \$/t**, plus an optional **proposed charter rate** left blank for auto-calculation.

Changing the origin region resets the destination to that region's first port and re-resolves the lane code. The resolved lane is echoed beneath the form — "Trade lane: Australia to Paradip · forecast uses the fixed synthetic historical series for AUS_PAR" — which is a deliberate transparency touch, since the lane code determines every downstream number.

12.3 Design system

Tailwind CSS v4 with a custom `@theme` token set. The palette maps the maritime domain directly: `navy-950` page background, `navy-800` card fill, `ocean-400/500/600` for primary data, and semantic accents `accent-green` (favourable), `accent-yellow` (caution), `accent-orange` (elevated), `accent-red` (adverse). Reusable `.card`, `.glow-blue` and `.glow-green` classes give the panel treatment. Icons are lucide-react (Ship, Anchor, TrendingUp, BarChart3, AlertTriangle, CloudRain, Clock, Activity, Ruler, PackageCheck).

13. End-to-End Worked Example

One request traced through all six modules, with every intermediate value.

13.1 The request

```
POST /api/full-analysis
{
  "cargo_quantity_tonnes": 120000,
  "cargo_material": "Coal",
  "route_code": "AUS_PAR",           // Newcastle → Paradip, 4,200 nm
  "contract_months": 6,
  "loading_date_month": 6,           // June
  "cargo_value_per_tonne": 120.0,
  "bunker_price_per_tonne": 580.0,
  "proposed_charter_rate": null      // → auto = 0.95 × breakeven
}
```

13.2 The chain of reasoning

#	QUESTION THE MODULE ASKS	ANSWER PRODUCED
1	Can any single hull carry 120,000 t into Newcastle and Paradip?	No. All 7 hulls fail gate G1 (95% usable capacity). Newcastle's LOA 250 m and beam 38 m independently eliminate both Capesizes. Optimizer falls to multi-voyage mode: the draft-limited capacity of a Panamax (Kamsarmax) on a 16.5 m limit is 77,900 t, so n = 2 voyages × 60,000 t . Kamsarmax wins on 0.45/0.30/0.15/0.10 split scoring (score 72.4). Alternatives: Panamax Std, Supramax Ultramax, Supramax Std, Handysize.
2	What will this lane cost over the next 90 days?	Trains the AUS_PAR ensemble on 1,364 rows (5.6 s). Last observed rate \$13.52/t . Day 1 forecast \$13.40 , day 3 \$13.36, flat thereafter. 95% band \$13.36–13.45 (d1) → \$12.09–14.63 (d30) → \$11.16–15.55 (d90). Validation MAE \$0.0357.
3	What is the true delivered cost of one 60,000 t voyage?	Freight \$804,000 + port \$228,000 + waiting \$184,908 + weather \$15,360 + insurance \$14,400 + bunker \$257,520 = \$1,504,188 per voyage. × 2 voyages = \$3,008,377 ⇒ \$25.07/t delivered against a \$13.40/t freight quote. The delivered cost is 87% higher than the freight line — and 46.5% of it is invisible in the quote.
4	Is there a cheaper feasible hull?	Prices all 5 feasible hulls at the 60,000 t per-voyage parcel (not at each hull's own maximum). Panamax Std and Supramax Std both at \$21.22/t, Kamsarmax and Ultramax at \$21.26/t, Handysize \$22.12/t — but the optimizer's Kamsarmax is retained , because selection_basis states feasibility and fit decide the vessel, and the cost spread is inside rounding. Correctly not overridden by a 0.2% price difference.
5	Can the Kamsarmax enter Paradip?	Loaded draft 12.76 m vs Paradip 18.5 m. Comfortable. DIRECT_BERTH , no lightering, no weather gate engaged — so weather_risk is null and the risk module falls back to its default weather score of 50.
6	Charter at \$12.69/t, or keep spot?	Breakeven \$13.36/t. NPV: charter \$4,464,488 vs spot \$4,699,965. CHARTER · HIGH confidence , P = 93.3%, expected saving \$235,478, VaR ₉₅ −\$27,058, worst case −\$368,654. Timing signal WAIT_FOR_WINDOW (current 7-day mean \$13.37 sits at the upper quartile).
7	How risky overall?	Volatility 25 · Weather 50 (default) · Congestion 100 ⇒ overall 52 · MODERATE . The binding risk is 11.6 days of berth waiting at Newcastle and Paradip, not the freight market.

13.3 The recommendation, assembled

PARCEL 120,000 T COAL · NEWCASTLE → PARADIP · JUNE LOADING · 6-MONTH CONTRACT

Ship: 2 × Panamax (Kamsarmax), 60,000 t each. Both Capesizes excluded by Newcastle berth geometry; all other hulls too small for a single voyage.

Cost: \$3,008,377 total, **\$25.07/tonne delivered** — against a \$13.40/tonne freight quote. Waiting time (\$7.37/t) and bunkers (\$4.29/t) are the two largest hidden lines.

Access: direct berth at Paradip, 5.7 m of draft to spare. No lightering, no weather exposure.

Decision: **CHARTER** at \$12.69/t (5% inside the \$13.36 breakeven). 93.3% probability of beating spot; expected saving \$235,478; downside capped at −\$27,058 in the worst 5% of outcomes.

Watch: port congestion scores 100/100 on 11.6 days of expected berth waiting. This — not the rate market — is the largest controllable risk in the plan.

13.4 Sensitivity of the answer to the inputs

CHANGE THE INPUT	EFFECT ON THE RECOMMENDATION
Material Coal → Steel	Same parcel selects a smaller hull (Supramax Ultramax at 55 kt). Material fit dominates the score at 0.40 weight.
Parcel 120 kt → 300 kt	Voyage count scales; per-voyage parcel and the cost total both grow, but cost/tonne is unchanged — the engine is correctly linear in parcel size once voyages are fixed.
Discharge port Paradip → Haldia	Dramatic. Draft limit 10.5 m forces Handysize and 5 voyages for the same 120 kt parcel. Cost/tonne rises on more voyages while the cargo physically cannot go any other way.
Loading month June → July	Weather-disruption cost rises (Paradip 0.20 → 0.25, Haldia 0.22 → 0.28) and, on a lightering voyage, the weather veto can flip the STS verdict outright.
Bunker \$580 → \$630/t	+ \$22,200 per voyage (+\$44,400 over two) — a 1.5% total-cost move from a single input, and the input most exposed to external market moves.
Proposed rate \$12.69 → \$13.80/t	Above breakeven. <i>P</i> (charter cheaper) collapses and the verdict flips to SPOT — the breakeven number is therefore directly usable as a negotiating limit.

14. Limitations & Validity Register

What this system genuinely demonstrates, what it does not, and what must change before deployment.

THE SINGLE MOST IMPORTANT SENTENCE IN THIS DOCUMENT

The engineering is real; the data is synthetic; the numerical outputs are therefore **demonstrations of mechanism, not statements about the freight market**. Nothing in Sections 5, 7 or 9 should be quoted as a market forecast, a price expectation or a risk quantification until the data layer is replaced with SAIL-approved historical series and the models are re-validated on them.

14.1 Data-layer limitations

#	ISSUE	CONSEQUENCE AND REQUIRED ACTION
D1	All history is synthetic	Five years of OU-generated rates with no true regime structure, no freight-market crises, no structural breaks. Action: ingest Baltic Exchange / Clarksons / SAIL fixture history. The forecaster's input contract makes this a data task, not a code change.
D2	Rate variance is far too small	AUS_PAR spans \$12.34–\$13.61 over five years ($\pm 2.5\%$). Real coal freight on this lane has moved by factors of 3–5. The narrow band is what makes MAE look excellent. Action: calibrate σ and the OU half-life to observed data; expect every accuracy metric to worsen by an order of magnitude, which would be more believable.
D3	Data is stale	The series ends 2024-12-29. As of this document's date that is roughly 21 months old, so even as a demo the "current rate" is not current. Action: regenerate with an end date near the present.
D4	Port figures are synthetic	Drafts, berth availability, handling rates, tariffs and weather probabilities are all plausible but unverified. Two visible tells: Sagar-Sandheads is listed under "India" with a 210 m LOA, and 21 of the 33 ports serve no lane. Action: replace with port authority / World Port Source data and re-derive the constraint tables.
D5	Data currency is decoupled from the model	<code>generate_synthetic_data.py</code> writes the CSVs but nothing invalidates a cached forecaster when the underlying file changes. Editing a feature file and re-requesting the same lane serves stale predictions until the process restarts. Action: key the cache on a file mtime hash.

14.2 Modules that are unreachable or dormant in the current configuration

#	ISSUE	CONSEQUENCE AND REQUIRED ACTION
M1	Canal fee logic is dead code	All 35 lanes have <code>canal_transit: null</code> , so the Suez and Panama tariff functions — including both full tiered schedules — can never execute. Verified: 0 of 35 lanes carry a canal. Correct for these origin/destination pairs (none routes via Suez or Panama), but it means ~25 lines of tariff logic are untested. Action: add one canal-transiting lane as a regression test.
M2	Weather risk inverts availability on 89% of lanes	The weather assessment lives inside the lightering module and only runs when lightering is required. On the 89% of combinations needing no transfer, <code>weather_risk</code> is <code>null</code> , so the risk module silently substitutes its default 50. Weather is never actually assessed for the voyage. Action: hoist weather assessment out of <code>lightering.py</code> into a port-level function called unconditionally.
M3	Lightering cost is excluded from total cost	The README claims nine cost components; the summation contains eight. On a lightering voyage the delivered-cost total understates reality by the whole STS operation (up to ~\$94k on the Capesize→Gopalpur case). Action: add <code>lightering_cost.total_cost_usd</code> to the total when the module recommends <code>LIGHTER</code> , and reconcile the README.
M4	Pie chart is inconsistent on split voyages	On a <code>FEASIBLE_SPLIT</code> result the cost total is multiplied by the voyage count but the breakdown rows are not. The table correctly prints a per-voyage note; the donut chart does not, so its slices sum to half the displayed total. Action: scale the pie's <code>dataKey</code> values by <code>voyage_plan.voyages</code> .
M5	7-day rolling mean degenerates on flat forecasts	When the forecast path is near-constant, the 25th and 75th percentiles of the rolling mean collapse to the same value, so the timing classifier is decided by floating-point noise and reports contradictory states. Observed on AUS_PAR: <code>WAIT_FOR_WINDOW</code> while simultaneously claiming the best window averages the same \$13.36 as every other window. Action: require a minimum relative spread (e.g. 0.5%) before emitting a directional signal, else return <code>MONITOR</code> .

14.3 Modelling limitations

#	ISSUE	CONSEQUENCE AND REQUIRED ACTION
MO1	Multi-step forecast is effectively flat	Only <code>rate_lag_1</code> is refreshed between recursive steps; the other 51 features are frozen at their last observed values, so the model reaches a fixed point by day 3. Verified: days 5–90 all return \$13.36. Action: implement a proper recursive feature-update loop that rolls every lag and recomputes the rolling statistics, or train a direct multi-horizon model per horizon. This is the highest-priority modelling fix in the system.
MO2	Low accuracy is an artefact of the data, not skill	Gain concentrates in <code>rate_rolling_max_7</code> (61.6%) and <code>rate_lag_1</code> (60.3%); BDI, bunker and all Fourier terms contribute ≈ 0 . The model learned that tomorrow $\approx$ today, which is near-optimal on a narrow OU series. The 0.274% MAPE is therefore not a claim about freight predictability. Action: report accuracy only after backtesting on real data, and publish a naive-persistence benchmark alongside it.
MO3	LSTM contribution is unverified and probably negative	Validation metrics are computed on the tree ensemble only ; the LSTM residual term is added at inference but never scored. It also receives a single time step at inference (<code>seq_len = 1</code>) despite being trained on 30-step sequences — a train/serve shape mismatch. Because a residual corrector with an untrained-on-real-data target can easily inject noise, it may be subtracting value. Action: score the full ensemble including the LSTM term, and either fix the sequence construction or drop the layer.
MO4	Monte Carlo ignores serial correlation	The 1,000 paths draw each month independently. Real freight rates are strongly autocorrelated, so true multi-month variance is materially larger than simulated and VaR is understated. Action: sample a correlated path — a multivariate normal with the empirical residual correlation matrix, or a bootstrap over historical monthly returns.
MO5	Monte Carlo is not reproducible	<code>monte_carlo_simulation()</code> calls <code>np.random.normal</code> with no seed. Two identical requests return different $P(\text{charter cheaper})$ values — observed 93.3% and 94.3% on the same input. Action: seed from the route code, or expose a seed in the request schema.
MO6	Breakeven is degenerate when the forecast is flat	With a constant forecast, R^* reduces to that constant — the discounting has nothing to differentiate. The weighted-average formula is correct; its input is currently uninformative. Action: resolves automatically once MO1 is fixed.
MO7	The 0.95 R^* default biases toward CHARTER	When no rate is proposed, the engine models a 5% discount to its own breakeven, so the charter case starts with a built-in advantage. Defensible as a negotiating posture, but it must be labelled as such in the UI rather than presented as a neutral finding. Action: make the default explicit in the response and in the panel copy.
MO8	No walk-forward or rolling backtest	A single chronological 80/20 split is the only evaluation. There is no multi-origin rolling-origin test, no per-regime breakdown and no comparison against a naive or ARIMA baseline. Action: implement expanding-window walk-forward evaluation across all 35 lanes and report per-lane MAE with a persistence baseline.

14.4 Engineering limitations

#	ISSUE	CONSEQUENCE AND REQUIRED ACTION
E1	Models retrain per request, in-request	Each uncached lane trains 5 XGBoosts, 1 LightGBM and 50 LSTM epochs synchronously inside the HTTP call — 6.55 s measured, blocking the event loop. Action: pre-train all 35 lanes at startup or in a background job and persist to disk.
E2	No persistence layer	Trained models live only in process memory. Every restart pays the full cold-start cost for every lane. Action: serialise with <code>joblib</code> / <code>torch.save</code> .
E3	No authentication, no rate limiting	All ten endpoints are open, including the model-training ones. Acceptable for a prototype on localhost; not acceptable on a shared network. Action: add auth and per-IP limits before any deployment.
E4	No tests	Zero unit or integration tests in the repository. Every numerical claim in this document was verified by ad-hoc execution. Action: add pytest coverage for the port-fee tariffs, the TPC integration against a closed-form check, and the NPV/breakeven identities.
E5	Top-level handler leaks tracebacks	<code>full_analysis</code> returns <code>traceback.format_exc()</code> in the 500 body, exposing file paths and code structure. Action: log the traceback server-side, return a correlation id to the client.
E6	Environment is not fully pinned	<code>requirements.txt</code> omits <code>packaging</code> and <code>setuptools</code> , which <code>torch.optim</code> imports transitively. On the bundled virtualenv this raised <code>ModuleNotFoundError: No module named 'packaging'</code> and the forecaster failed at the LSTM step until both were installed. Action: add both to <code>requirements.txt</code> — this is a one-line fix that prevents a confusing first-run failure.
E7	Reference tables re-read from disk per call	<code>routes.json</code> , <code>vessels.json</code> and <code>ports.json</code> are parsed on every function call — <code>recommend_vessel</code> reads all three, and <code>compare_vessel_options</code> calls back into it. Action: load once at import with <code>functools.lru_cache</code> .
E8	No containerisation or CI	No Dockerfile, no CI workflow. Setup is a shell script plus manual npm install. Action: add a Dockerfile and a CI job that runs the generator, boots the API and asserts the response schema.

14.5 Scope gaps against the problem statement

- **Alternative employment is asserted, not computed.** §9.6 recommends evaluating another trade lane during soft windows, but no module actually searches the other 34 lanes for a better netback. This is the most valuable missing capability and a natural next module: for each idle window, maximise `rate - voyage_cost` across all reachable lanes.
- **No ballast or repositioning model**, so idle-management cost is invisible and the recommendation is qualitative.
- **No inventory or lay-time integration** — the model assumes cargo is ready when the ship arrives.
- **No multi-leg or transshipment routing.** Every lane is direct, so the option of moving via a transshipment hub (which is how several of these cargoes actually move) is absent.
- **No emissions or CII dimension.** Fuel is costed but not carbon-costed; increasingly material to charter decisions.
- **No counterparty or credit assessment.** Charterer quality is outside the charter/spot framing entirely.

14.6 Summary: what the prototype legitimately demonstrates

✓ Established

- A complete, inspectable, end-to-end decision chain with no black boxes.
- Constraint-first vessel selection that correctly refuses physically impossible ships, including multi-voyage planning.
- Genuine quantile-regression confidence intervals with correct $\sqrt{h}$ horizon widening — not symmetric error bars.
- A delivered-cost model showing that freight is only ~53% of the decision.
- A TPC-integrated lightering calculation that fires precisely where bathymetry requires it, gated by real OCIMF weather thresholds.
- A reproducible Monte Carlo with VaR, breakeven and an explicit probability of the recommendation being right.
- A monsoon-aware weather model that reproduces real East Coast Indian seasonality.

! Not yet demonstrated

- Any predictive accuracy claim — all error metrics are artefacts of the synthetic generator.
- A working multi-step forecast — the central path is flat beyond day 3.
- A reliable charter/spot probability — currently an arithmetic consequence of three favourable defaults.
- Realistic multi-month risk — Monte Carlo ignores serial correlation and understates VaR.
- Lightering economics inside the delivered cost.
- Weather risk assessed on voyages that need no lightering (89% of cases).
- Computed alternative employment for idle windows.
- Production readiness: no auth, no tests, no persistence, no CI.

14.7 Prioritised remediation plan

PRIORITY	ACTION	UNBLOCKS
P0 • Blocking	Add <code>packaging</code> and <code>setuptools</code> to <code>requirements.txt</code>	First-run success on a clean environment (E6)
P0 • Blocking	Replace the synthetic series with SAIL-approved history; recalibrate σ , the OU half-life and the volatility clustering	Every accuracy and risk claim (D1, D2, MO2)
P1 • High	Implement true recursive multi-step feature updating, or train direct per-horizon models	A non-flat forecast path, meaningful breakeven, meaningful timing signals (MO1, MO6, M5)
P1 • High	Hoist weather assessment out of the lightering module into an unconditional port-level call	Weather risk on 89% of lanes (M2)
P1 • High	Add lightering cost into the delivered-cost total; fix the split-voyage pie chart	Cost completeness and chart correctness (M3, M4)
P2 • Medium	Correlated Monte Carlo paths + a request seed	Honest VaR and reproducible results (MO4, MO5)
P2 • Medium	Score the LSTM term properly; fix the sequence mismatch or drop the layer	Knowing whether the third ensemble member helps (MO3)
P2 • Medium	Pre-train and persist all 35 models; cache JSON tables at import	Cold-start latency and throughput (E1, E2, E7)
P3 • Lower	Add <code>pytest</code> , a <code>Dockerfile</code> and a CI schema assertion	Maintainability and deployability (E4, E8)
P3 • Lower	Build the alternative-employment module across all 35 lanes	Closing the largest scope gap (§14.5)

15. Deployment, Reproduction & References

15.1 Prerequisites

REQUIREMENT	VERSION	NOTES
Python	3.9+	Developed and verified on 3.11. The bundled <code>backend/venv</code> is 3.11.
Node.js	18+	Vite 6 requires 18+.
Disk	~400 MB	Predominantly the virtualenv (PyTorch alone is ~250 MB).
First run	~60–90 s	Dependency install plus data generation; ~30 s of that is the generator.

15.2 One-command start

```
# Linux / macOS
./run.sh          # creates venv, installs deps, generates data, starts both servers

# Windows
run.bat
```

The script detects the Python interpreter (3.11 → 3.10 → 3.9 → 3), creates `backend/venv` if absent, installs with `CMAKE_POLICY_VERSION_MINIMUM=3.5`, generates the dataset only if `data/freight_rates.csv` is missing, then starts Uvicorn on `:8000` and Vite on `:5173` with a `trap` that kills both on Ctrl-C. Open <http://localhost:5173>.

15.3 Manual start

```
# Terminal 1 – backend
cd backend
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
python data/generate_synthetic_data.py      # ~30 s → 39 CSVs
uvicorn main:app --reload --port 8000

# Terminal 2 – frontend
cd frontend
npm install
npm run dev
```

KNOWN FIRST-RUN ISSUE (E6)

On a clean install, `torch.optim` fails with `ModuleNotFoundError: No module named 'packaging'` while building the LSTM optimiser state. Fix with `pip install packaging setuptools`, or add both to `requirements.txt` as the remediation plan specifies. Everything else runs clean.

15.4 Generator output

```
Generated market_indicators.csv: 1825 rows
Generated freight_rates.csv: 1825 rows, 35 routes
Generated features_AUS_PAR.csv: 1705 rows, 57 features
... 35 files total ...
Data generation complete!
```

Deterministic under `np.random.seed(42)`: regenerating on any machine reproduces byte-identical files, which is what allows the figures in this document to be independently checked.

15.5 Verifying the platform without the UI

```
# Health and data check
curl http://localhost:8000/api/health
# → {"status":"healthy","timestamp":"...", "data_generated":true}

# Full analysis (first call trains the model, expect ~6 s)
curl -X POST http://localhost:8000/api/full-analysis \
-H "Content-Type: application/json" \
-d '{"cargo_quantity_tonnes":120000,"cargo_material":"Coal","route_code":"AUS_PAR",
    "contract_months":6,"loading_date_month":6,"cargo_value_per_tonne":120,
    "bunker_price_per_tonne":580,"proposed_charter_rate":null}'

# Interactive API docs at http://localhost:8000/docs
```

15.6 Data sources and reference frameworks consulted

SOURCE	USED FOR
Baltic Exchange (C5, C3, P1A, S10)	Route structure conventions and the shape of published freight indices — the structure is modelled; the values are not Baltic data.
Clarksons Research	Fixture-data patterns and the structure of historical time-charter and voyage data.
World Port Source / MarineTraffic	Port draft limits, LOA/beam constraints, berth rates and charge schedules — the categories modelled in ports.json .
NOAA GFS	Marine weather forecast methodology behind the monthly delay-probability construct.
IMO / OCIMF	Ship-to-ship transfer guidance, including the $H_s > 1.5$ m suspension threshold used as the hard weather veto in §8.4.

15.7 Academic references

- Alizadeh, A. H. & Nomikos, N. K. (2009). *Shipping Derivatives and Risk Management*. — OU and GARCH specification for freight rate processes; the charter-vs-spot derivative framing used in §9.
- Kavussanos, N. M. & Visvikis, V. V. (2006). *Derivatives and Risk Management in Shipping*. — mean reversion in freight rates and the volatility-clustering evidence motivating the GARCH state.
- Stopford, M. (2009). *Maritime Economics*, 3rd ed. — delivered-cost decomposition, freight-market structure, charter-versus-spot trade-off.
- Rawson, K. & Tupper, E. (2001). *Basic Ship Theory*, 5th ed. — hydrostatics, the TPC concept and the waterplane-area basis of §8.
- Friedman, J. H. (2001). "Greedy Function Approximation: A Gradient Boosting Machine." — XGBoost.
- Ke, G. et al. (2017). "LightGBM: A Highly Efficient Gradient Boosting Decision Tree." — LightGBM.
- Hochreiter, S. & Schmidhuber, J. (1997). "Long Short-Term Memory." — the residual-correction network.
- Hull, J. (2018). *Options, Futures and Other Derivatives*. — NPV, Value-at-Risk and the Monte Carlo framing of §9.3.
- Newey, W. & West, K. (1987). "A Simple, Positive Semi-Definite, Heteroskedasticity and Autocorrelation Consistent Covariance Matrix." — the autocorrelation caveat underlying §14.3 MO4.